QCon
全球软件开发大会

InfoQ 技术大会

Agent 驱动的全平台 GUI 持续验证实践

从 MCP 辅助录制到独立 Agent 的工程演进



Windows　macOS　Android　iOS　Web

演讲人：Zhengda Wang

GUI 自动化的长期门槛



1 编写门槛高

需要熟悉并掌握自动化框架

2 平台能力割裂

业务验证要求跨平台覆盖，但框架与平台能力并不天然跨平台

3 录制语义弱

传统录制只记录操作，缺少断言、等待条件和业务意图

4 脚本维护重

UI 改动、控件变化、异步加载、环境差异都会导致脚本失效

GUI 自动化的复杂性不只在“写脚本”，而是分散在用例表达、代码生成、跨平台执行和维护的完整链路中。

AI Coding 与 MCP 带来的新机会



GUI 自动化既需要生成测试代码，也需要操作真实 GUI 环境 —— 这两件事恰好被同时补齐。

把 GUI 自动化能力封装成 MCP，让 AI Coding 既能生成脚本，又能操作真实 GUI。

第一阶段演示



第一阶段：MCP + VS Code 插件辅助录制

BDD Case → VS Code 插件 触发录制 → VS Code Copilot → 平台 MCP → 真实 GUI → 自动化脚本 → 人工确认

01 VS Code 插件

提供从 Case 发起录制的入口

02 平台 MCP

提供真实 GUI 的操作能力

03 VS Code Copilot

调用 MCP 完成录制，并生成自动化脚本

```
@p0
Automated | Send to Copilot | Run
Scenario: Use copilot to summarize webpage
  When I click search box on the page
  And I enter "https://en.wikipedia.org/wiki/Ultraman_Leo" in the search box
  And I click go to search
  And I click Copilot button on the address bar
  And I clicked the "Create a Summary" button on the Copilot chat page
  Then I should see a summary of the page in the Copilot dialog
```

插件入口：从 BDD Case 发起录制

```
    assert result_json.get("status") == "success", f"Expected status to be 'success', got '{result_json.get('status')}', error: '{result_json.get('error')}'"

# --- auto-generated step ---
@step('I enter "https://en.wikipedia.org/wiki/Ultraman_Leo" in the search box')
def step_impl(context):
    result = call_tool_sync(context, context.session.call_tool(
        name="send_keys",
        arguments={'caller': 'behave-automation',
            'locator_strategy': 'AppiumBy.NAME',
            'locator_value': 'Address',
            'need_snapshot': 0,
            'text': 'https://en.wikipedia.org/wiki/Ultraman_Leo'}
    ))
    result_json = get_tool_json(result)
    assert result_json.get("status") == "success", f"Expected status to be 'success', got '{result_json.get('status')}', error: '{result_json.get('error')}'"

# --- auto-generated step ---
```

最终产物：生成的可执行自动化代码

VS Code 插件 + MCP 把 GUI 自动化接入 AI Coding 工作流，明显降低从零写脚本的成本。

两层约束提升录制稳定性

BDD Case

约束任务表达

- 明确前置条件
- 明确操作路径
- 明确断言目标
- 减少 AI 自行补步骤、改步骤、跳步骤

Plugin Prompt

约束执行方式

- 指定使用 MCP，要求按步骤执行
- 要求每一步调用 MCP tool
- 不修改、合并、跳过或新增步骤
- 通过截图 / snapshot 查看界面信息

Please use native-mcp-server to execute the following instructions:

Original BDD Test Case (strictly follow step-by-step):

Scenario: Start a new topic with copilot
Given I sign in to edge with MSA
When I click Copilot button on the NTP page
And I enter "Hello" in the input box
And I click the send button
And I click the new topic button in the upper right corner of the header
Then I should see a Copilot dialog with no dialog information

Requirements:
feature_file =
'c:\Users\yuexiong\projects\FSQ_AI_Testcases_iOS\features\copilot\chat.feature'

1. Before executing the first step, call `before_gen_code`, and after all steps are completed, sequentially call `preview_code_changes` and `confirm_code_change`.
2. Execute **each step** exactly as written using native-mcp-server API calls, in order.
3. **MANDATORY: For EVERY step, you MUST call an MCP tool. Never judge results by screenshots/snapshots alone.**
4. **Do not modify, merge, skip, or add any step.** Do not lose bdd step keyword.
5. Do not close browser

插件封装的 Prompt：Case 内容 + MCP 使用约束

BDD Case 让任务表达更明确，Plugin Prompt 让执行方式更受控，两层约束共同提升录制稳定性。

第一阶段数据效果

5.01M

Test Steps

近30 天执行步骤数
Windows 1.35M / Android 1.19M / iOS 584K / macOS 472K

6,964

Hours Saved

近 30 天节省工时

736

Automated Cases

Android 196 / iOS 192 / Windows 189 / macOS 159

200+

Bugs Found

自动化用例发现的产品缺陷

4

Platforms Support

Windows / macOS / iOS / Android

99.7%

Pass Rate Consistency

接近零 flaky 的通过率一致性

近 30 天四个平台共执行 5.01M 测试步骤、节省 6,964 工时，累计发现 200+ 缺陷，验证了 AI 录制生成代码 + 回放这条路径的可行性。

第一阶段是怎么跑起来的

日常运行：每天都在跑

Azure Pipeline 每日执行

录制好的用例代码在 Azure Pipeline 上 daily 跑，产出当天结果

结果随新增 Bug 一起 Review

每天 review 当天新增 bug 时，一并看自动化结果，分清真 bug 与过期用例

过期用例当天重录

判定为 case 过期的，需要重新录制后才能回到可用状态

瓶颈：录制稳定性与维护仍依赖人工

ID 变了就得重录

产品发布后 flow 没变，只是控件 id 改了，用例照样失效

维护量随规模放大

录制仍要人全程盯着，用例越多，重录和维护占掉的人力就越重

录制 Harness 不够稳定

录制过程只靠 Prompt 约束，Copilot 仍会跳步、走偏，需要人工盯着和返工

GUI 自动化天然是 Agent 任务

目标驱动

围绕明确的测试任务持续推进

上下文理解

理解自然语言 Case、项目知识、历史 Case 与当前界面状态

任务规划 / 编排

把测试目标拆成可执行步骤，组织录制、执行与反馈流程

GUI 自动化 Agent

工具调用

调用 MCP / Driver / 平台能力，对真实 GUI 产生操作

Harness 支撑

提供稳定、可控、可预期的执行过程

反馈循环

依据界面状态与执行结果调整后续动作

Agent 适合承接 GUI 自动化：把“辅助录制”扩展为围绕测试任务持续执行并沉淀资产的工程流程。

GUI 自动化不是一次性的脚本生成，而是围绕测试任务持续推进的 Agent 任务。

从辅助录制到 Agent 闭环

第一阶段：MCP 辅助录制		第二阶段：独立 Agent 闭环
自动化代码 依赖 AI Coding 生成	产物 →	**DSL Case** Agent 生成更友好的可执行结构
Prompt 约束 主要依靠 Prompt 约束录制过程	稳定性 →	**Harness 执行保障** 由更完善的 Harness 承接执行稳定性
人工确认 人工确认录制过程和代码可用性	验收 →	**Evidence + Agent 判断** Evidence 支撑 Agent 自动判断
BDD 格式 Case 需要较严格的 BDD 格式	门槛 →	**自然语言 Case** 使用人群不再局限于测试角色

Agent 使执行过程更稳定，参与门槛更低，测试得以更贴合项目 Context 持续演进。

第二阶段：独立 Agent 系统



Adapter Layer 输入输出适配层	CLI	Web Frontend	Agent Protocol		
Application Layer 应用服务层	LLM Provider Auth	Case Management	Run Result		
	Workspace	Environment	Config		
Agent Layer 智能体层	Flow Orchestration	Pre-planning	Prompt Context	Verification Strategy	
Execution Layer 执行核心层	Case	Runner	Evidence		
	Harness	Loop	Report		
Driver Layer 平台驱动层	Android	Web	Windows	macOS	iOS

输入更自然

自然语言 Case 进入系统，不再强依赖 BDD 格式

生成可执行结构

Agent 结合上下文生成 / 更新 DSL Case

执行与证据闭环

Harness 调度 Driver，Evidence 回流判断与修复

第二阶段由独立 Agent 编排完整链路，从 Case 到执行结果全部接管，而不只是生成代码。

第二阶段运行流程：录制与回放

录制



回放

without LLM



录制阶段把自然语言 Case 转成 DSL Case，回放阶段 without LLM 执行，结果可复现、可追踪。

为什么坚持录制回放

01

结果可重复，录一次反复回放

严格按录制下来的步骤回放，每次执行路径完全一致；录制成本只付一次，之后每轮回归都直接复用

02

失败判定更可信

跑挂了就是产品的 bug，不用再分辨是不是 LLM 规划走偏，去掉结果的不确定性

03

更快、更便宜、可高频

回放不经过 LLM，没有推理等待，也不消耗 token，可以用更高的频率触发

DSL Case：把自然语言意图转成可执行结构

DSL Case

```yaml
schemaVersion: fsq.ai-test/v1
name: copilot-disable-excerpt
description: >
  Disable Copilot from settings, verify the Copilot
  icon disappears, then enable it again.
platform: android
appId: com.microsoft.emmx
---
- tapOn:
    target: Enable Copilot new tab page toggle
- assert:
    element:
      resourceId: com.microsoft.emmx:id/switchWidget
      checked: false
```

示例节选：关闭开关与状态断言；导航、图标检查及恢复步骤省略。

1 **承接原始意图**

保留自然语言 Case 描述，对应原始业务目标

2 **结构化回放**

固化步骤、定位、断言和平台信息，回放按 DSL 执行

3 **直观可审查**

操作与断言直接可读，便于理解和检查

4 **稳定可迭代**

可版本化、可 diff，支持随产品变化重新生成

DSL Case 连接自然语言意图与确定性回放。

DSL 的生成与回放机制

示例 Step Flow

① 进入设置页 → ② 关闭 Copilot 开关 → ③ 检查关闭状态 → ④ 恢复开关

录制

AI 参与

AI 根据 Case 和当前界面，确定动作与参数

Harness 执行流程

Before → driver.tap_on (params) → After

对应的 DSL

录制：tap_on → tapOn
保留目标参数

```
- tapOn:
    target: Enable Copilot new tab
page toggle
```

回放

without LLM

由 DSL 确定方法与参数，无需 AI 再次选择

Harness 执行流程

Before → driver.tap_on (params) → After

Before 获取执行上下文，采集操作前截图和 UI Tree

Driver 方法 根据目标参数定位控件并点击

After 按配置等待，采集操作后截图和 UI Tree，记录执行结果

录制固化动作与参数；回放解析后调用同一 Driver 方法，前后处理由执行框架统一完成。

可插拔 Driver：承接不同平台能力差异



1 平台能力各异

页面结构、控件定位、输入方式和应用生命周期随平台变化。

2 录制与回放共用能力

AI 调用和 DSL 执行接入同一套平台能力，避免分别维护两套实现。

3 新增平台复用上层

通过 Driver 接入平台后端，复用 Case 格式、执行调度和证据链路。

统一 Case 格式和执行接口，由 Driver 适配平台后端差异。

Driver 能力设计：观察、操作、断言、反馈

1

Observe

观察

判断当前界面和操作目标，需要可用的界面信息。

截图、UI Tree、窗口或页面状态

2

Act

操作

将测试步骤落实到真实 GUI，需要可调用的操作能力。

点击、输入、滚动、快捷键

3

Assert

断言

判断预期是否成立，需要显式检查结果。

文本、可见性、控件状态

4

Feedback

反馈

判断是否继续及定位失败，需要明确的执行反馈。

执行状态、错误类别、结果信息

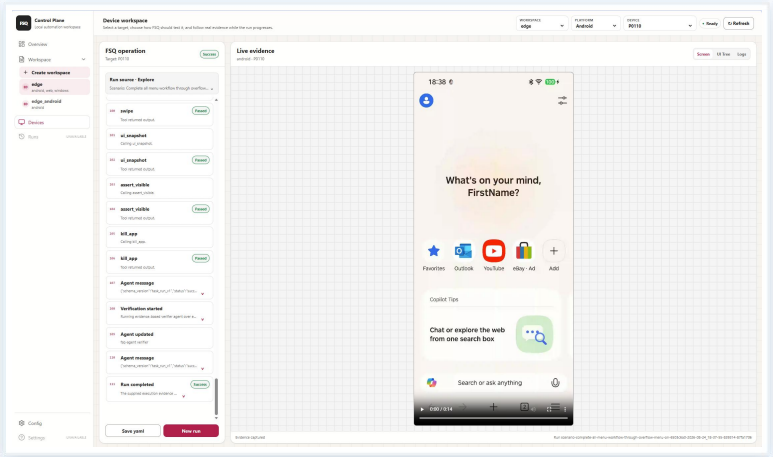
平台实现

Web　Android　Windows　macOS　iOS

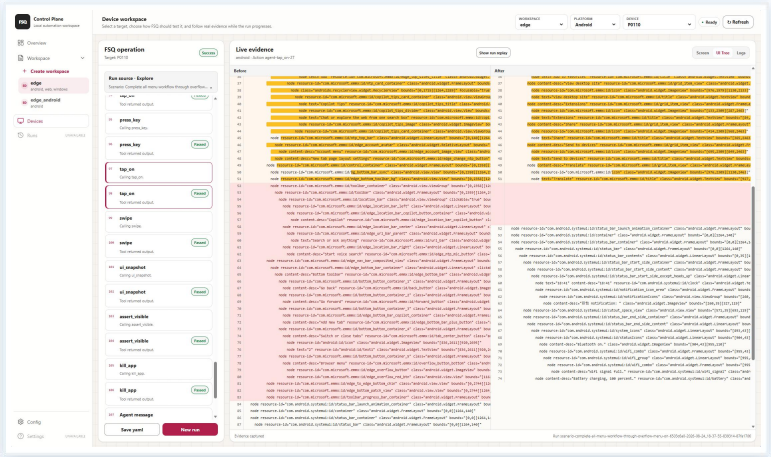
MCP 阶段积累的观察、操作、断言与反馈能力，在第二阶段形成统一的 Driver 能力设计。

Evidence：沉淀执行过程与问题现场

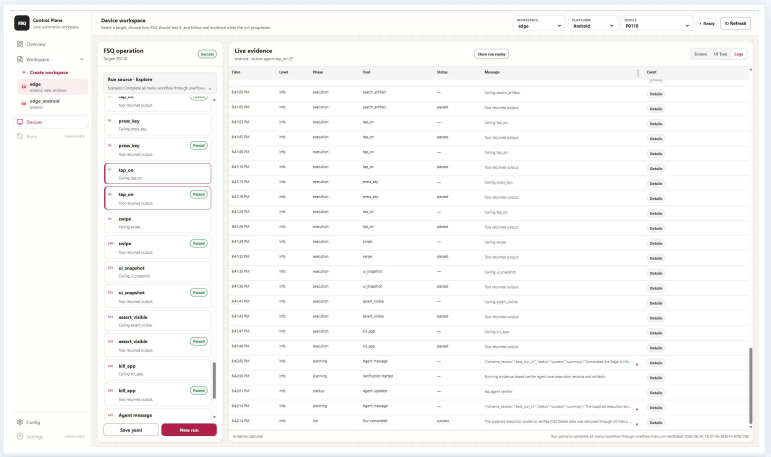
Case Run → Step Trace → Evidence Bundle → Review / Replay / Debug



Video：还原 GUI 操作路径



UI Tree Diff：观察界面结构变化



Log：定位 tool 调用与异常信息

步骤轨迹

执行状态 / 输入输出 / 失败位置

过程回放

视频还原 GUI 操作路径

界面证据

截图 / UI Tree / UI Tree Diff

执行日志

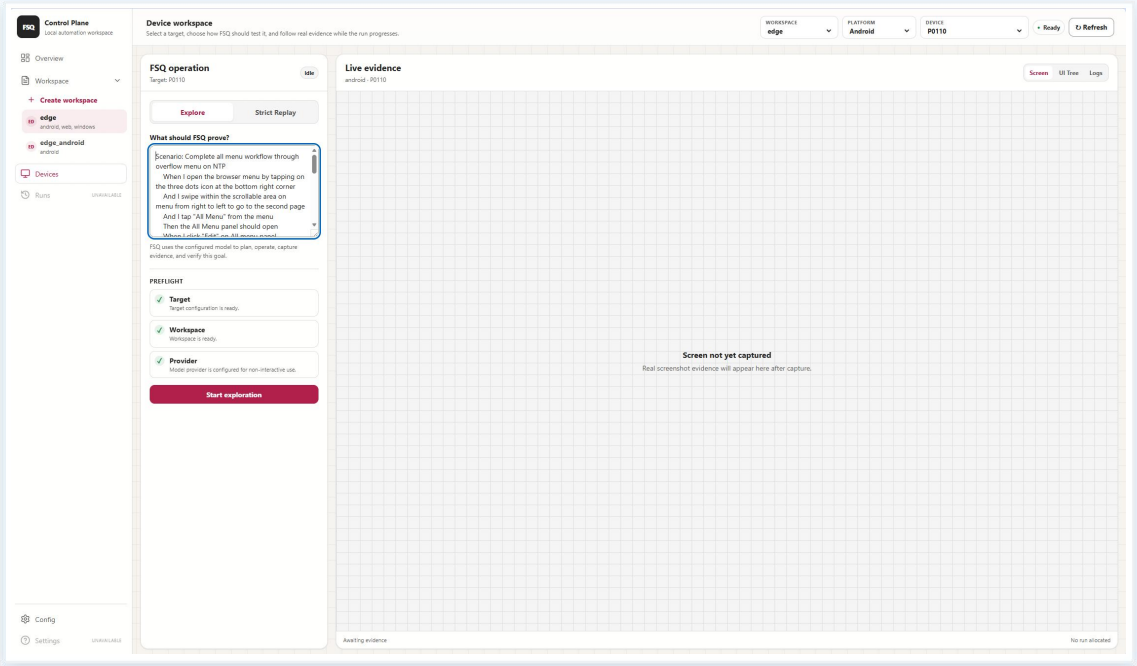
tool 调用 / 运行日志 / 异常

结果判断

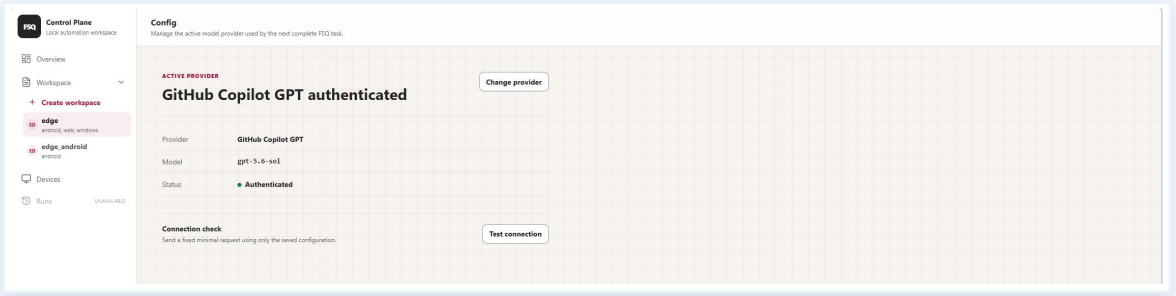
支撑 key action / key assert

Evidence 把一次执行沉淀为可回看的问题现场，成为 Agent 判断与人工诊断的共同事实。

多入口使用形态：Web / CLI / Agent Protocol



Web Frontend：创建 Case 并跟随 Live Evidence



Web 同时承载 Provider 配置与授权认证

CLI

```
fsq-agent control-plane
fsq-agent init --name web-demo --platform web
fsq-agent run --platform web --record --goal "..."
fsq-agent run --platform web --strict --case-yaml ...
```

Web

日常使用：Case 管理、运行查看与 Provider 授权

CLI

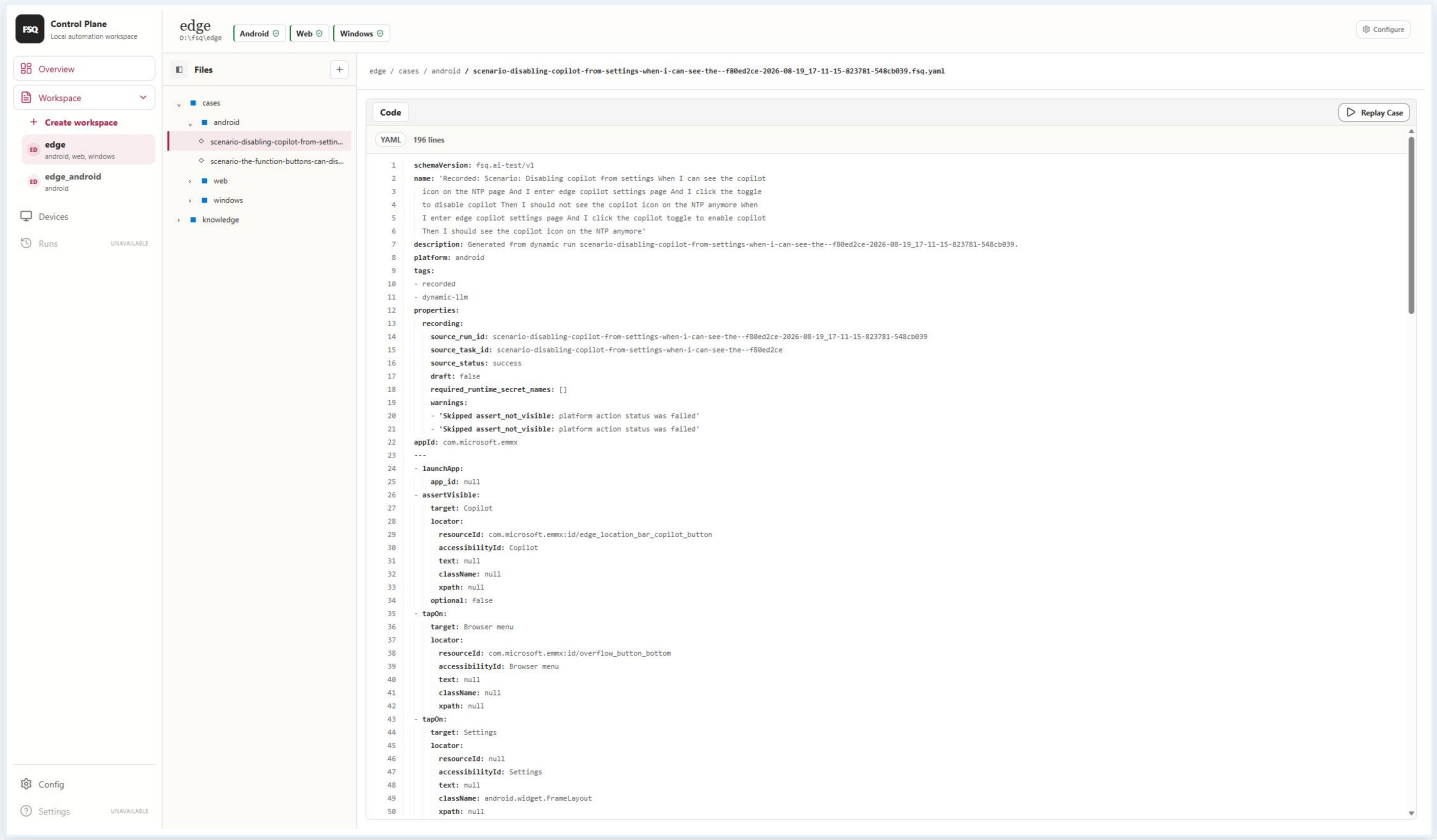
自动化与 CI：能力与 Web 对齐，适合脚本与批量执行

Agent Protocol（规划中）

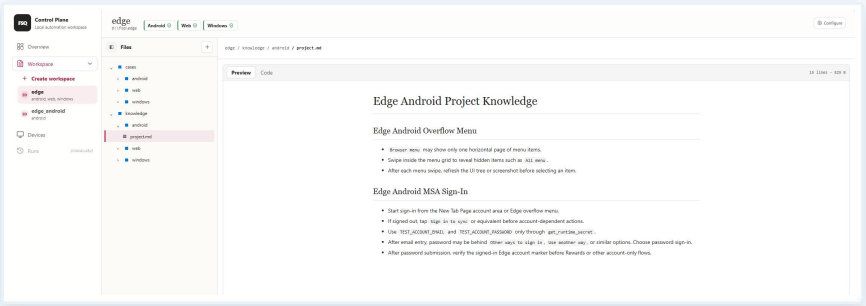
外部接入：用于对接其他 Agent 或自动化系统

同一套 Agent 能力多入口输出：Web 降低门槛，CLI 支撑集成，Agent Protocol 面向协作。

Case 管理与项目 Context



Case 管理页：workspace / case 列表 / case 内容三栏结构



项目私有知识配置

自然语言 Case → 项目 Context → Agent → DSL Case

Workspace 组织

按项目、平台或应用组织 Case

Case 管理

集中查看、创建和编辑自然语言 Case

项目 Context

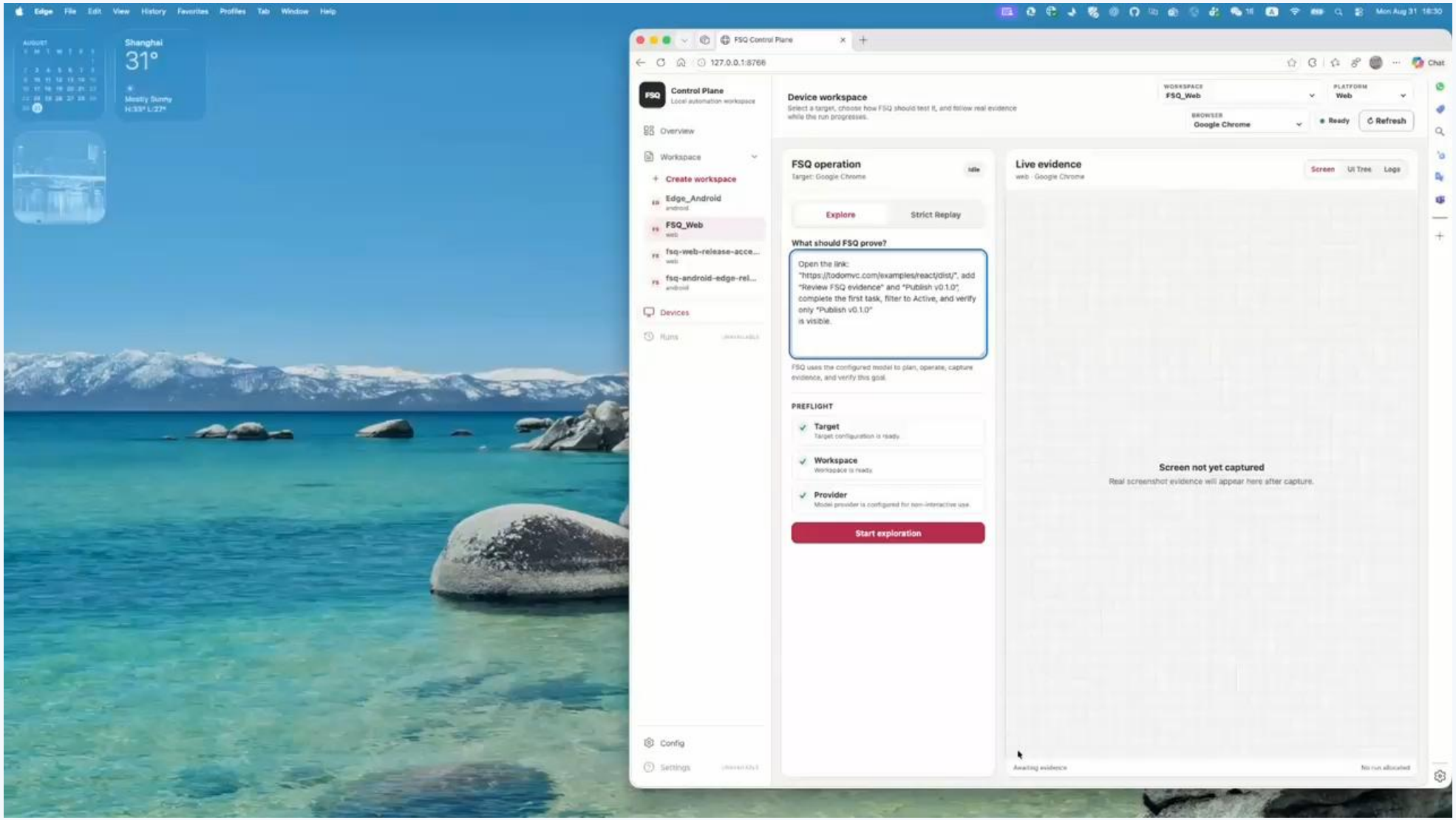
沉淀业务规则、术语和环境约束

Agent 使用

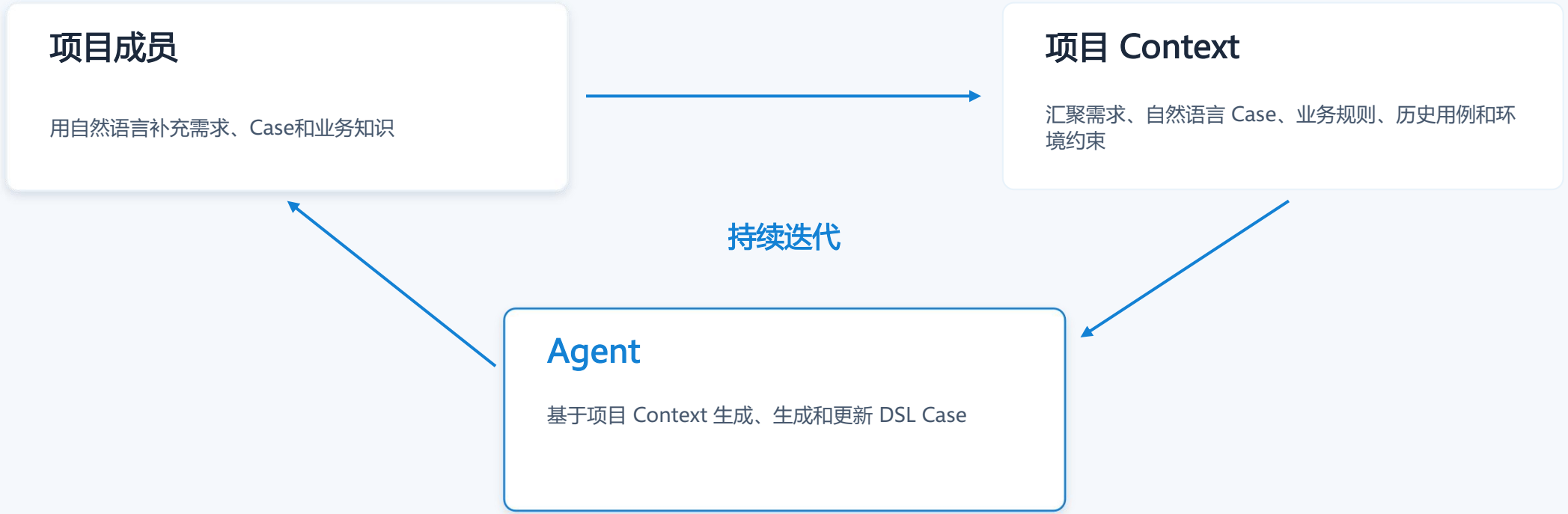
结合 Case 与 Context 生成 DSL Case

人用自然语言表达验证意图，项目 Context 补足业务与环境知识，Agent 生成可执行 DSL Case。

第二阶段演示


Control Plane
Device workspace
Overview
Workspace
Create workspace
Edge_Android
FSQ_Web
Devices
Runs
Config
Settings
FSQ operation
Explore
Strict Replay
What should FSQ prove?
Open the link: "https://todomvc.com/examples/react/dist/", add "Review FSQ evidence" and "Publish v0.1.0", complete the first task, filter to Active, and verify only "Publish v0.1.0" is visible.
PREFLIGHT
Target
Workspace
Provider
Start exploration
Live evidence
Screen not yet captured
Real screenshot evidence will appear here after capture.
Refresh
Ready

未来展望：围绕项目Context持续迭代



项目成员提供需求

所有项目成员都能参与验证意图的表达

Context 沉淀项目知识

业务规则与环境约束不再散落在个人经验里

Agent 持续生成和优化

自动化能力随项目一起演进，而不是一次性交付

从按角色分工写自动化，走向围绕项目 Context 共同驱动 Agent 的持续迭代能力。

AiCon 北京站

全球人工智能开发与应用大会

探索 AI 应用边界

Explore the limits of AI applications

2026 年 12 月 18–19 日 / 北京新云南皇冠假日酒店

参会咨询

专题：Post-Training Engineering | 数据、奖励与模型能力塑造



专题出品人：余瑶

飞猪 / 资深算法专家

专题：大模型推理与算力调度实践



专题出品人：范苑

阿里云 / 专有云计算产品负责人

专题：Knowledge Engineering | 让 Agent 理解企业



专题出品人：盛茂家

火山引擎 / Viking 团队技术负责人

专题：Loop & Graph Engineering



专题出品人：孙子荀

腾讯 / 腾讯高级总监

专题：AI 原生架构的演进与治理



专题出品人：刘中兵

快手 / 架构治理团队 & AI 架构演进负责人

专题：Agentic Software Engineering | 从代码生成到软件交付



专题出品人：杨杰

荣耀 / 首席 IT 技术架构师

QCon
全球软件开发大会

InfoQ 技术大会

THANKS

Agent 驱动的全平台 GUI 持续验证实践 · 从 MCP 辅助录制到独立 Agent 的工程演进

MCP 辅助录制 → 独立 Agent 系统



演讲人：Zhengda Wang

项目链接：FSQ